TEACHER VERSION Same booklet as the pupils get, plus the 90 minute timeline (this page) and the answers in red. Do not hand out.

Timeline (90 minutes)

Short talks then short tasks, most of the time in code. Slide numbers are those in the pptx (timing pills and speaker notes are on the slides). "Booklet p" means the page of the pupil booklet; in this version the answers can push content onto the next page.

Minutes	Activity	Slides	Booklet / notes
0–10	**Talk.** What do you see? Python sees numbers (think, pair, share). Libraries and PIL. A picture is a grid.	1–9	Pupils open Spyder and `image_explore.py`.
10–15	**Task 1:** open a picture, predict the shape.	10	Booklet p2
15–18	Talk: rows and columns.	11	
18–21	**Task 2:** find a pixel.	12	Booklet p2
21–23	Talk: crop (slicing).	13	
23–26	**Task 3:** crop a corner.	14	Booklet p3
26–30	Talk: three numbers per pixel (RGB).	15, 16, 18	Skip 17 (paper sheet).
30–33	**Try it:** mix a colour.	19	Booklet p3
33–35	Talk: split the channels.	20	
35–38	**Task 4:** split the channels.	21, 22	Booklet p4
38–40	Talk: switch a channel off.	23	
40–43	**Task 5:** switch channels off.	24, 25	Booklet p4
43–45	Talk: grey and negative.	26	
45–48	**Task 6:** grey and negative.	27	Booklet p5
48–50	Talk: maths on the picture.	28	
50–53	**Task 7:** count and average.	29	Booklet p5
53–55	Talk: histograms.	30, 31	
55–58	**Task 8:** draw a histogram.	32, 33	Booklet p6
58–63	**Talk, Part 2:** the puzzle and the rule (open, add the red, % 26, letter).	36–39	Booklet p8
63–65	Talk: why we need code. Two different photos.	41, 42	Skip 40 (be the computer is optional).
65–68	Talk: STEP 1 and STEP 2, open `image_starter.py`.	43, 44	Booklet p9
68–73	**Code:** `total_red`. Test: 10218857.	45 (leave up)	Hints in booklet p9
73–75	Talk: STEP 3, `letter_for`.	46	
75–80	**Code:** `letter_for`. Test: Z.	46 (leave up)	Booklet p9
80–83	Talk: STEP 4, `image_path`, building a string with +=.	47–49	
83–88	**Code:** `decode_message`. Does it read like English?	50, 51	Booklet p10 tracker
88–90	Recap and reflect.	54, 55	

Realistic expectations. It is tight. Part 1 ends at 58 minutes so Part 2 has 32 minutes. If the room is slow, cut the Try it task and the histogram talk first. **Optional or stretch only** (not taught from the front): slides 17, 34–35, 40, 52–53, the stretch pages, `decode_message` for the slowest third.

Image Analysis Challenge

One booklet for the whole lesson: instructions, hints and your tracker.

Name: _____ Date: _____

What you will do today. A photo is just a grid of numbers. You will use Python to open one, look inside it, change it and measure it (Part 1). Then you will use what you learned to decode a hidden message from 37 photos (Part 2).

How this booklet works. Your teacher will talk, then you get a short task, then the teacher talks again. If you finish a task early, **carry on to the next one**. Everything you need is in here. Do not wait for the class.

Hints come in two levels. Try it yourself first. If you are stuck, read **Hint 1**. Still stuck? Read **Hint 2**. The answer is never printed, because you can work it out.

Set up Before you start

- ☐ Open **Spyder**.
- ☐ Find the folder with the lesson files. It must contain `image_explore.py`, `image_starter.py`, `cow-demo.jpg` and a folder called `cow_message`.
- ☐ Open `image_explore.py`. Run the whole file with **F5** (green triangle). Run just a few lines by selecting them and pressing **F9**.
- ☐ Pictures appear in the **Plots** pane. Numbers appear in the **Console**.

The `import` lines load libraries (code someone else wrote): **PIL** opens pictures, **NumPy** (`np`) does maths on grids of numbers, **matplotlib** (`plt`) draws. Leave them alone.

Tracker Tick each one when it works

Part 1	**in image_explore.py**	
Task 1	Open a picture	☐
Task 2	Find a pixel	☐
Task 3	Crop a corner	☐
Try it	Mix a colour	☐
Task 4	Split the channels	☐
Task 5	Switch channels off	☐
Task 6	Grey and negative	☐
Task 7	Count and average	☐
Task 8	Draw a histogram	☐
Part 2	**in image_starter.py**	
Step 1 and 2	Add up the red in one photo	☐
Step 3	Turn one photo into a letter	☐
Step 4	Decode the whole message	☐

Task 1 Open a picture — 5 min

In `image_explore.py`, find **TASK 1**. It opens `cow-demo.jpg` and turns it into a grid of numbers called `pixels`.

Predict first. `pixels.shape` gives three numbers. What do you think each one counts?

They count the rows (how tall), the columns (how wide) and the numbers for each pixel (red, green, blue).

Now run the file. The three numbers I got: **(320, 320, 3)**

So the three numbers are: rows **320** columns **320** ??? **3 (R, G, B)**

Hint 1

Look at the picture in the Plots pane. How tall is it? How wide? The third number is something you will meet again soon.

Hint 2

`shape` lists the numbers in this order: (up and down, across, then the extra number for each pixel).

Task 2 Find a pixel — 3 min

Rows count down. Columns count across. Counting starts at 0, in the top left corner. The pixel in row r and column c is `pixels[r, c]`.

1. Run the line that prints the pixel in row 0, column 0. I got: **[16 0 0]**
2. **TODO:** print the pixel in row 100, column 200. I got: **[94 50 37]**
3. Predict before you run it: which of the three numbers will be the biggest?

Teacher: model answer

`print(pixels[100, 200])` gives `[94 50 37]`. The first number (red) is the biggest, so it is a reddish brown. Row 0, column 0 is `[16 0 0]`, almost black.

Hint 1

Use the same shape as the line above it. Row goes first, then column.

Hint 2

```
print(pixels[ ___ , ___ ])
```

Task 3 Crop a corner — 3 min

A crop uses a **slice**: `pixels[row start:row stop, column start:column stop]`. It stops *before* the stop number. So `0:100` means rows 0 to 99.

1. Run TASK 3 to see the top left corner.
2. **TODO:** crop a different 100 by 100 square, then show it.
3. **Challenge:** can you crop just the cow's head?

My crop used rows __100__ to __200__ and columns __100__ to __200__.

Teacher: model answer

```
square = pixels[100:200, 100:200]
show(square)
```

Any sensible 100 by 100 slice is fine. For the head, any window around the upper middle of the cow. Common mistake: swapping rows and columns.

Hint 1

Change the four numbers. Keep the colon and the comma.

Hint 2

```
square = pixels[ ___ : ___ , ___ : ___ ]
show(square)
```

Try it Mix a colour — 3 min

A pixel is three numbers: how much **red**, **green** and **blue** light, each from 0 (none) to 255 (the most). In the file, find `show_colour(230, 30, 30)`. Change the three numbers and run it.

TODO: make yellow. R __255__ G __255__ B __0__

TODO: make a grey. R __128__ G __128__ B __128__

Your own favourite colour: R __any__ G __any__ B __any__

Hint 1

Light mixes the other way to paint. Red light and green light together look yellow. For grey, think about how the three numbers compare.

Hint 2

Yellow has two high numbers and one low number. Grey has all three numbers the same, somewhere between 0 and 255.

Task 4 Split the channels — 3 min

Each pixel has three **channels**: red (0), green (1) and blue (2). `pixels[:, :, 0]` means every row, every column, just channel 0.

1. Run TASK 4. You see the red channel. Bright means *lots of red*.
2. **TODO:** do the same for green and for blue.
3. Which channel is brightest on the wall? **red** On the cow? **all dark**

Teacher: model answer

```
green = pixels[:, :, 1]
show_channel(green, "Greens")
blue = pixels[:, :, 2]
show_channel(blue, "Blues")
```

Hint 1

Copy the red lines. Change the channel number and the colour name (`"Greens"` , `"Blues"`).

Hint 2

```
green = pixels[:, :, ___ ]
show_channel(green, "Greens")
```

Task 5 Switch channels off — 3 min

`.copy()` makes a second picture so the original stays safe. Then set one whole channel to 0:

```
no_red = pixels.copy()
no_red[:, :, 0] = 0
show(no_red)
```

Predict first. What colour will the picture go with no red? With no green? With no blue? Write or sketch it:

No red: blue-green (cyan). No green: pink-purple (magenta). No blue: yellow-brown. Accept any reasoned prediction.

1. **TODO:** make a picture with no green, and one with no blue.
2. **TODO:** make a picture that is *only* blue (red and green both 0).
3. Was your prediction right? **their own: check it against the picture**

Teacher: model answer

```
no_green = pixels.copy()
no_green[:, :, 1] = 0
show(no_green)
no_blue = pixels.copy()
no_blue[:, :, 2] = 0
show(no_blue)
only_blue = pixels.copy()
only_blue[:, :, 0] = 0
only_blue[:, :, 1] = 0
show(only_blue)
```

Hint 1

Copy the three `no_red` lines. Change the name and the channel number. For “only blue” you need to switch off two channels.

Hint 2

```
no_green = pixels.copy()
no_green[:, :, ___ ] = 0
show(no_green)
```

Task 6 Grey and negative — 3 min

`img.convert("L")` gives one brightness number per pixel (0 black, 255 white). The negative is `255 - pixels` : every number flips.

Predict first. The first pixel was `[16 0 0]` . What will it be in the negative?

`[239 255 255]`, because $255 - 16 = 239$, $255 - 0 = 255$ and $255 - 0 = 255$.

1. Run TASK 6. First pixel of the negative: **[239 255 255]**
2. What happened to the black cow? **it turns pale (nearly white)**
3. How many numbers does each pixel have in the grey picture? **1**

Hint 1

Everything for this task is already written in the file. Run it and look at both pictures. To predict the first pixel, work out `255 - 16` , `255 - 0` and `255 - 0` .

Task 7 Count and average — 3 min

You can do maths on the whole picture. `grey.mean()` is the average brightness. `(grey < 50).sum()` counts how many pixels are darker than 50.

1. Run TASK 7. Average brightness: **81.45**
2. Pixels darker than 50: **28501**
3. **TODO:** how many pixels are *brighter* than 100? **34315**
4. **TODO:** what fraction of the whole picture is that? **0.335 (about a third)**
5. Predict first: more than half of the picture, or less? **less than half**

Teacher: model answer

```
print((grey > 100).sum())
print((grey > 100).sum() / grey.size)
```

Hint 1

Change the `<` and the number in the line above it. For the fraction, divide by the total number of pixels: `grey.size` .

Hint 2

```
print((grey ___ ___ ).sum())
print((grey ___ ___ ).sum() / grey.size)
```

Task 8 Draw a histogram 3 min

A histogram splits the brightness values into buckets and counts how many pixels fall in each bucket. A tall bar means lots of pixels with that brightness.

1. Run TASK 8. Where are the tallest bars: left (dark), middle or right (bright)? **left (dark)**
2. **TODO:** draw a histogram of just the red channel. Then green. Then blue.
3. What is different about the three? **red leans right, green and blue lean left**

Sketch the red histogram here (just the rough shape):

Red: tall bars in the middle to right (the bright wall). Green and blue: tallest bars on the left. The grey histogram has a dark hump on the left (cow and street) and a second hump on the right (the wall).

Teacher: model answer

```
plt.hist(red.flatten(), bins=16)
plt.show()
```

Then the same with `green` and `blue`.

Hint 1

The grey histogram line is already written. Use the same idea with the `red` picture you made in Task 4.

Hint 2

```
plt.hist( ___ .flatten(), bins=16)
plt.show()
```

Stretch Part 1: finished early?

1. Go backwards. `::-1` means "go backwards". In the file, find the STRETCH block. Run `show(pixels[:, ::-1])` and `show(pixels[::-1])`. Which is the mirror and which is upside down?

`pixels[:, ::-1]` is the mirror. `pixels[::-1]` is upside down.

2. Brighten it. Add 60 to every number. Careful: what happens above 255? A pixel was 200. What do you think it becomes? Try it and see.

$200 + 60 = 260$ wraps round to 4 (a byte only holds 0 to 255), so bright pixels turn dark.

3. Think. In your own words, why is a photo "just numbers"?

A photo is a grid of pixels and each pixel is three numbers (red, green, blue), so the whole picture is a block of numbers.

Part 2 Hidden in plain sight

Cryptography hides what a message *says*. **Steganography** hides that there is a message *at all*. A photo is a good place: it is millions of numbers, and changing one of them by 1 is invisible.



200, 80, 60 201, 80, 60

Pixels as (red, green, blue). The right one has red changed by 1. You cannot see the difference. A program can read it.

A People really do this

year	what happened
2010	Russian agents living in the USA hid messages inside ordinary pictures on public websites. The FBI found out.
2017	A Russian hacking group, Turla, hid instructions in the comments under a pop star's Instagram photo. Security researchers at ESET worked out the trick.
2022	Malware was hidden inside a real picture from the James Webb Space Telescope. Security researchers at Securonix spotted it.

B How researchers find hidden messages

- **Look at the numbers.** A normal photo has a pattern in its histogram (you drew one in Task 8). Hidden data can make it look odd.
- **Compare with the original.** Subtract one picture from the other. Any change shows up.
- **UK research.** Researchers at the University of Cambridge's Computer Laboratory wrote some of the first papers on hiding information, and showed that messages can hide inside ordinary internet traffic.
- **UK defence.** The National Cyber Security Centre, part of GCHQ, helps protect the UK from hidden threats.
- **History.** In World War 2, British censors checked letters for invisible ink and hidden writing.

C Your scenario

Imagine you work for a security team. A website is posting **37 photos of cows**. Nobody needs 37 cow photos… unless they are a message. Each photo hides **one letter** in its pixel numbers. Your job: work out the rule, then decode all 37 letters.

Part 2 The puzzle: how it works

37 photos are hidden in the folder `cow_message`. Each one hides **one letter**. Together the 37 letters spell a message. The rule is the same for every photo:

1. Open the photo.
2. Add up every **red** value in it. This is the **total**.
3. Take the total `% 26`. That means "the remainder after dividing by 26", so it is a number from 0 to 25.
4. Use that number to pick a letter: A is 0, B is 1, C is 2, and so on until Z is 25.

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

Part 2 Be the computer (optional paper warm-up)

Do image 1 first, then 2 and 3 if you want to feel what the computer does. Add up the nine red numbers, take the total `% 26` (keep taking 26 off until you cannot), then find the letter.

Image 1 (worked example)

60	58	146
179	176	119
164	185	85

Image 2

48	212	202
35	126	187
177	61	162

Image 3

44	180	113
178	98	155
165	160	47

	Total of the 9 numbers	**Total % 26**	**Letter**
Image 1	1172	2 ($1172 - 26 \times 45$)	C
Image 2	**1210**	**14 ($1210 - 26 \times 46$)**	**O**
Image 3	**1140**	**22 ($1140 - 26 \times 43$)**	**W**

How long? One tiny image took me **about 1** minutes. The real photos are 640 by 640, which is 409 600 pixels each, and there are 37 of them. Roughly how long by hand? **over 3 years**

So what should we get the computer to do?

Write functions that open a photo, add up the red, take % 26 and pick the letter, then loop over all 37 photos. About $409600/9 \approx 45\,511$ tiny images per photo, times 37, is about 1.68 million minutes.

Step 1 Look at a photo

Open `image_starter.py`. **STEP 1 is done for you.** Run it: it prints the size, the shape and the first pixel of `cow-demo.jpg`, then draws a colour.

Careful: `cow-demo.jpg` is our **320 by 320** practice photo. The 37 puzzle photos in `cow_message` are **640 by 640**. The same code works on both.

Step 2 Add up the red in one photo 5 min

Finish the function `total_red(path)`. It opens the photo at `path` and returns the sum of every red value. The test line should print **10218857** for `cow-demo.jpg`.

```
def total_red(path):
    img = Image.open(path)
    pixels = np.array(img)
    # TODO: get just the red channel
    # TODO: add up every red value in it
    # TODO: return that total
```

Teacher: model answer

```
red = pixels[:, :, 0]
total = red.sum()
return total
```

Test line prints `10218857`.

Hint 1

You already got one channel in Task 4. You already counted with `.sum()` in Task 7. The function must `return` the answer, not just print it.

Hint 2

```
red = pixels[:, :, ___ ]
total = red.___ ()
return ___
```

Test fails? If it prints `0`, you forgot to change `return 0`. If the number is different, check the channel number.

Step 3 Turn one photo into a letter 5 min

Finish `letter_for(path)`. It gets the red total, takes it `% 26`, and uses the result to pick a letter from `ALPHABET`. The test line should print **Z** for `cow-demo.jpg`.

Remember: `ALPHABET[2]` is `"C"`. You wrote a function in Step 2 that does the first part.

Teacher: model answer

```
total = total_red(path)
position = total % 26
letter = ALPHABET[position]
return letter
```

Test line prints `Z` ($10218857 \bmod 26 = 25$).

Hint 1

Call the function you wrote: `total_red(path)`. Then `%` gives the position (0 to 25). Square brackets pick the letter.

Hint 2

```
total = ___ (path)
position = total % ___
letter = ALPHABET[ ___ ]
return ___
```

Step 4 Decode the whole message 5 min

Finish `decode_message()`. For each photo number from 1 to 37 it should work out the letter and stick it on the end of `message`. `image_path(number)` is given to you: `image_path(1)` gives `"cow_message/01.png"`.

```
def decode_message():
    message = "" # the message we are building. It starts empty.
    # TODO: for each number from 1 to 37
    # 1. find the picture: image_path(number)
    # 2. work out its letter
    # 3. stick the letter on the end of message: message += letter
    return message
```

Then remove the `#` from the last line of the Step 4 block to run `print(decode_message())`. It can take a few seconds. **Does it read like English?** Then it worked. If it is gibberish, check Step 2 and Step 3 again.

Teacher: model answer

```
for number in range(1, 38):
    path = image_path(number)
    letter = letter_for(path)
    message += letter
```

Hint 1

`for number in range(1, 38):` counts from 1 to 37 (it stops before 38). Inside the loop: get the path, get the letter, add the letter with `+=`.

Hint 2

```
for number in range(1, 38):
    path = image_path(number)
    letter = ___ (path)
    message += ___
```

Tracker Write each photo's letter

1 T	2 H	3 E	4 A	5 T	6 T	7 A	8 C	9 K	10 W
11 I	12 L	13 L	14 T	15 A	16 K	17 E	18 P	19 L	20 A
21 C	22 E	23 A	24 T	25 T	26 E	27 N	28 P	29 M	30 S
31 A	32 T	33 U	34 R	35 D	36 A	37 Y			

The message says:

THE ATTACK WILL TAKE PLACE AT TEN PM SATURDAY

Stretch Part 2: finished early?

1. A different channel. Copy your functions, change the 0 to a 1 (green) or a 2 (blue), and run it. What message do you get? Does it make sense?

Green and blue give nonsense messages. For `cow-demo.jpg` the green total is 7707223 and the blue total is 6674243.

2. Think. If one photo's red total went up by 1, which letter would come next? What if it went up by 26?

Up by 1: the next letter in the alphabet. Up by 26: the same letter, because the remainder is the same.

3. Make your own function. Write one function `total_channel(path, channel)` that adds up *any* channel you choose, so you do not need to copy your code. Test it on `cow-demo.jpg`.

Write your code here

Teacher: model answer

```
def total_channel(path, channel):
    pixels = np.array(Image.open(path))
    return pixels[:, :, channel].sum()
```

4. Off the computer. Explain to a friend how a computer can "see" a photo. Use the words *pixel*, *channel* and *number*.

A photo is a grid of tiny squares (pixels). Each pixel is stored as three numbers, one for each colour channel (red, green, blue), from 0 to 255. The computer only ever sees the numbers.